Project: Service Mesh & Observability

Due: 04/18/2026 11:59 PM
Extended Deadline: 04/19/2026 11:59 PM

Introduction
- Objectives
- Context & Prerequisites
- Evaluation Method
- Handout

Istio
- Task Overview
- Task

Prometheus
- Task Overview
- Task

Open Telemetry
- Task Overview
- Task

Wrap Up
- Conclusion

Open Telemetry (Task)

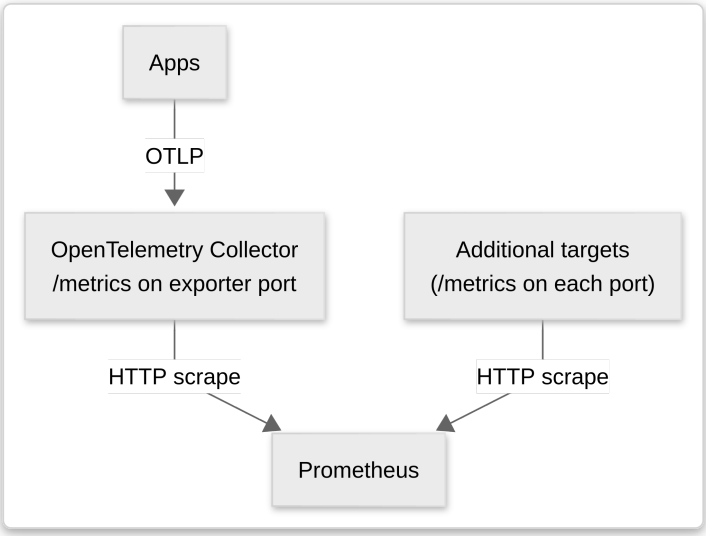
Task 3: Further Instrumentation

Now that we have finished fixing our system, let's go and ensure that our entire system is sending all relevant metrics to Prometheus. This will allow any analyst or admin to see everything they may want to know about the system, and so long as the metrics do not drastically reduce latency it is generally considered best practice to add metrics when possible.

This is going to be a fairly involved process - as a reminder, we need to add logging to the following components:

1. Both of our codebases
2. MariaDB
3. Our Istio Gateway

Let's start with our codebases, as they will require the most work to integrate.



Information

Add any new yamls in the observability directory.

Subtask 1: FastAPI Instrumentation.

The term "instrumentation" here simply refers to the act of adding metrics to an existing system. Instrumenting any system in order to provide good metrics is an art form more than a science, requiring a fair amount of experience with your system to specify correctly. As a result, while you can create your own metrics for Prometheus to ingest, we will instead instrument both of our services through what is known as "automatic instrumentation", where we use an external library to handle sending and developing metrics instead of writing our own.

The general best-practice for this is to use OpenTelemetry. This is an initiative to both standardize instrumentation across different systems and standardize metrics SDKs for microservice developers. OpenTelemetry supports exporting data in a format that Prometheus can ingest, and it also allows us to use systems like Zipkin and Jaeger for distributed tracing.

Information

If you have completed Project 3 Task 2, you may wonder why we are using the SDK as opposed to a service mesh like Istio. While this is partially so that you can see the more manual way of handling metrics without a service mesh, as you may sometimes need to work without one for reasons like security or the additional overhead a mesh provides, we would also like to stress that the metrics that you get from a service mesh will not necessarily include everything that you may want to know about your application. The benefits of this approach are that we will get the best of both automating metrics and language / framework specific instrumentation.

For our LLM service, we will use OpenTelemetry's Operator to both:

1. Automatically instrument application-specific metrics for our system.
2. Collect them into a single location, and relay them to Prometheus.

In reality, this is pretty close to what KrakenD is doing implicitly - it exports to OpenTelemetry and then moves it to Prometheus afterwards - but now we will need to implement it, as we are unable to use what KrakenD does directly.

Firstly, we need to install yet another Kubernetes Operator - in this case we want to install the OpenTelemetry operator:

```
kubectl apply -f https://github.com/cert-r

# Verify cert-manager pods
kubectl get pods -n cert-manager

kubectl apply -f https://github.com/open-t
```

Once installed, you can check that the operator pods are running with:

```
kubectl get pods -n opentelemetry-operato
```

If both containers are running, you will get something like the following:

```
NAME
opentelemetry-operator-controller-manager-
```

Now that we have the operator installed, you can probably guess that we have new CRDS to write. Moreover, unlike the MariaDB CRDS we glanced over during the setup, in this case we are going to go through them in detail, so that you can feel comfortable about what's going on.

The first resource you will make is the "OpenTelemetryCollector" resource. This is effectively a data lake / storage area for your analytical data, before it gets sent over to Prometheus. You could connect this collector with other backends, like Zipkin for tracing, but for now we will just focus on collecting metrics with OpenTelemetry, sending them to the Collector, and forwarding them to Prometheus.

Let's walk through an example, courtesy of the OpenTelemetry documentation:

```
apiVersion: opentelemetry.io/v1alpha1
kind: OpenTelemetryCollector
metadata:
  name: my-collector
spec:
  mode: deployment # This configuration is
  config: |
    receivers:
      jaeger:
        protocols:
          grpc:
    processors:

    exporters:
      debug:

    service:
      pipelines:
        traces:
          receivers: [jaeger]
          processors: []
          exporters: [debug]
```

On a high-level, this config is particularly strange as the developers behind the Operator wanted the spec to basically just be a transparent configuration between the YAML configuration for the collector, and the Kubernetes YAML. It's particularly strange as it makes everything under "config" unchecked by the operator itself, so you will only get error messages from the operator's pods if you miss something or run into an error.

Besides that strange piece of design, however, there are effectively two parts to the `spec`:

mode, which is the kind of deployment the underlying pods should use. Currently this is set to a Deployment, but the operator has support for DaemonSets and StatefulSets as well.
config, which is just the YAML config in its entirety.

In any OpenTelemetry Config, you need to specify four main sections:

1. **The Receivers:** These are the particular places our OpenTelemetry system can get logs, metrics, and

traces from. In this example, OpenTelemetry takes in logs from an older system that uses `Jaeger` instead for data storage and processing.

2. **The Processors:** These allow you to do some minimal processing stages, like slowing down the rate at which the pods send data to the next group, and also to control and send more data at once.
3. **The Exporters:** These are how OpenTelemetry actually sends data to the rest of your services. Currently, the export is a simple debug exporter, which is nice if you need to print out what is being sent over, and you want to know if OpenTelemetry is receiving the packets.
4. **The Pipeline:** These are the different paths your data can go through from input, to processing, to output. You can think of OpenTelemetry like a really really simple Spark Job - A multi-stage process that takes in some data, potentially does some processing, and sends the output forwards.

For this part of the task, you will need to modify the above spec to make it functional with your existing Prometheus system. While Prometheus plans to have better support for Open Telemetry in the next few years, currently it takes a fair bit of effort to get them to talk to one another. Therefore, we will give an outline of what you will need to do:

1. You will need to change the mode from "deployment" to a "statefulset". Prometheus exporting only supports a statefulset deployment strategy, and so we do not have any flexibility there.
2. For your receivers, you will need to add an `otlp` reciever. This must support both `grpc` and `http` protocols, and you must specify the `http` protocol to use the `endpoint` 0.0.0.0:4318. This ensures that the Open Telemetry system is reachable all throughout our cluster, making it readily accessible to everyone.
3. For your processors, you will need to add a `batch` processor. We suggest setting `send_batch_size` to `10000` and `timeout` to `10s`, but we will not grade these settings.
4. For your exporters, you will need to add a `prometheus` exporter, pointing to `0.0.0.0:9464` in particular. As Prometheus is pull-based as

opposed to push-based, all we need to do here is specify a particular port that we will be able to access the OpenTelemetry Collector instead of specifying how to reach Prometheus directly. Said another way, this is just client-side discovery as opposed to server-side discovery; Prometheus is the Collector's client, and it looks for data when it wants to ingest more data and can.

5. Besides the `prometheus` exporter, we also suggest adding a `debug` exporter as well, and setting `verbosity` to `detailed`. This is especially important, as we want to get some reasonable error messages in case we run into any trouble.
6. In terms of the pipeline, you should specify a single pipeline with `otlp` as the only receiver, `batch` as the only processor, and `prometheus` (and potentially `debug`) are your exporter(s). The pipeline configuration keys are important; you may need to adjust them.
7. Lastly, but most importantly, you must name your collector `simplest`, for grading purposes.

Once done, run `kubectl apply -f YOUR_YAML_FILE` to install the collector.

Interim Validation

If you did this properly, you should see a pod called `simplest-collector-0` when you run `kubectl get pods`, and if you look at the logs you should see something close to the following:

```
2024-04-07T19:03:20.332Z        info    se
2024-04-07T19:03:20.333Z        info    se
2024-04-07T19:03:20.333Z        info    ex
2024-04-07T19:03:20.334Z        info    se
2024-04-07T19:03:20.334Z        info    ex
2024-04-07T19:03:20.334Z        warn    in
2024-04-07T19:03:20.334Z        warn    in
2024-04-07T19:03:20.335Z        info    ot
2024-04-07T19:03:20.335Z        warn    in
2024-04-07T19:03:20.335Z        info    ot
2024-04-07T19:03:20.335Z        info    se
```

Automatic Instrumentation with an Instrumentation Resource

The second CRD we will provision is the `Instrumentation` resource. This is a system that does the following:

1. Tells the operator to look for any deployment with a particular `annotation` attribute.
2. When it sees a deployment with that attribute, it will hijack the pod and add an initContainer to it, starting the instrumentation pipeline directly.
3. The initContainer will figure out what framework the web service is running, and then provide the correct instrumentation for that situation.

While you lose the ability to make your own metrics, this kind of deployment, where all you need to use is an annotation, is fairly popular among service meshes like LinkerD, as well as any other system that acts as middleware. It's particular hands-off for developers, so when it works, it **really** works. On the other hand, when it does not work, it can be a bit painful to debug.

Here is an example instrumentation resource:

```
apiVersion: opentelemetry.io/v1alpha1
kind: Instrumentation
metadata:
  name: simplest-instrumentation
spec:
  exporter:
    endpoint: ADDME
  propagators:
    - tracecontext
    - baggage
  sampler:
    type: parentbased_traceidratio
    argument: "1"
```

In this case, this example is taken almost verbatim from the documentation, but this is specifically a resource that ensures **Python** programs can be successfully instrumented automatically.

Before we dive into what is going on, let's think aloud for what such a system might require. Even if it sounds like utter magic (imagine having such a debugger easily runnable with minimal effort anywhere), we know that such a system will need:

1. Someplace to know where to send the data. While it is part of the operator, Cloud Operator designers want to ensure that each YAML resource contains everything necessary for its function, as that is just good API design. In our case, this is the `exporter`.
2. Some format to send the data. There are usually many ways one can send data like trace data to someone else, and so we generally want to agree upon any existing standard. The `propagators` determine the standard and enforce data coming out for tracing to follow the W3C's standards for tracing, Baggage and Trace. While we will not cover tracing in this project, it is easily addable to this task.
3. Some way to limit some of the data - particularly for stuff like tracing, where we could have the data for every microservice communication across several channels. In our case, this is the `sampler`.

All you need to do for this section is to add the `endpoint` of our system. This is just the internal Kubernetes address we need to use to connect to the `simplest-collector` service, using `http://`, at port `4318`, which is the gRPC ingestion port.

Interim Validation

Assuming you defined this as expected, if you run `kubectl get instrumentation`, you will get the following:

```
NAME                        AGE   ENDPOINT
simplest-instrumentation    57m   http://si
```

You can also confirm that the operator understood your configuration through:

```
kubectl describe otelinst
```

This resource should only get created once the instrumentation resource has been created. In particular, it will contain the following to indicate that the system can instrument Python code:

```
instrumentation.opentelemeti
  ghcr.io/open-telemetry/ope
```

Updating our Deployment

Now that the hard configuration is done, we have a relatively simpler operation - we just need to update our llmserver's deployment YAML.

Under `spec.template.metadata`, add a key called `annotations`, and add the following to that key:

```
instrumentation.opentelemetry.io/inject-py
```

After doing that, you should be done, but this is a particularly tricky thing to get 100% right. Remember that you should add the deployment again for the llmserver.

Interim Validation

To check that you did do this correctly, you have a few choices:

1. The end-to-end validation.

You can curl the `simplest-collector` pod at port `9464`, specifically under `/metrics`, after confirming that your application still works by curling the LLM service using an access token. This is the exact route Prometheus will use to get the current metrics. If you did it correctly, you will get any non-empty response - something that will look like the following:

```
# HELP asyncio_process_created_total Numbe
# TYPE asyncio_process_created_total count
asyncio_process_created_total{job="llmserv
# HELP asyncio_process_duration_seconds Du
# TYPE asyncio_process_duration_seconds hi
asyncio_process_duration_seconds_bucket{jo
asyncio_process_duration_seconds_bucket{jo
asyncio_process_duration_seconds_bucket{jo
asyncio_process_duration_seconds_bucket{jo
asyncio_process_duration_seconds_bucket{jo
asyncio_process_duration_seconds_bucket{jo
asyncio_process_duration_seconds_bucket{jo
asyncio_process_duration_seconds_bucket{jo
asyncio_process_duration_seconds_bucket{jo
asyncio_process_duration_seconds_bucket{jo
asyncio_process_duration_seconds_bucket{jo
asyncio_process_duration_seconds_bucket{jo
asyncio_process_duration_seconds_bucket{jo
asyncio_process_duration_seconds_bucket{jo
```

```
asyncio_process_duration_seconds_bucket{jo
asyncio_process_duration_seconds_bucket{jo
```

To use `curl` with the simplest-collector, you can launch a temporary container with a Bash shell using a command like:

```
kubectl run curl-pod --rm -it --restart=Ne
```

Once inside, you can execute your `curl` requests, like the following:

```
curl http://simplest-collector:9464/metric
```

1. A more indirect approach

You could also check that the initContainers on the provisioned `llmserver` pod contain the `opentelemetry-auto-instrumentation-python` container as well. The easiest way to do this is by running something like:

```
kubectl describe pod llmserver-b57bc4c7f-6
```

Making sure that you get something close to the following:

```
Annotations:      instrumentation.opentele
  opentelemetry-auto-instrumentation-pytho
    Image:          ghcr.io/open-telemetry/
    Image ID:       ghcr.io/open-telemetry/
      /otel-auto-instrumentation-python fr
      PYTHONPATH:
      /otel-auto-instrumentation-python fr
  opentelemetry-auto-instrumentation-pytho
```

We are almost at the home stretch of configuring this particular sub-system! To finish things off, we just need to get Prometheus to know about the new data source.

Configuring Helm Charts, and Getting Prometheus to Talk.

If you remember the setup instructions, you may recall that we did not really use `kubectl apply` to install Prometheus. Throughout the course, we generally were content to just install a helm chart, and then use CRDs

or other systems to configure the pods around the chart to work with the charted resource.

In the case of Prometheus, we will need to do it another way. This is because Prometheus has no dynamic (see: post-deployment) way to add new data sources; as it is real-time, the developers decided to generally make that something you configure ahead of time, at least for cases unlike KrakenD where we did not need to do anything to get it working.

In this case, this will be through creating a `values.yaml` file. In general, Helm allows you to customize installations of various systems through one of two options:

1. Command line arguments
2. YAML files

In general, if you can get away with a command line argument (i.e. the amount of changes is not sufficiently large or multiline), you probably should configure your Helm charts through command line arguments. In our case, the parameter we want to change, **additionalScrapeConfigs**, is a multi-line YAML string, so it really does not make sense to try to format a command to fit that.

For our case, our **values.yaml** file will look like the following:

```
prometheus:
  prometheusSpec:
    additionalScrapeConfigs: |
      - job_name: "prometheus-otel-exporte
        static_configs:
        - targets: ["ADDME"]
```

Walking you through, the first two lines are essentially boilerplate as the Helm chart provisions non-Prometheus systems for various stability and performance reasons, but all we are doing is specifying:

1. **A job name**, which is just the same metrics source information that we talked about in the first part.
2. **A static config**, which in our case just contains a single string containing the **target**, or the hostname that we want to connect to.

In general, whenever you want to customize a Helm chart in this manner, you can do the following:

1. Get the complete list of options through:

```
helm show values prometheus-community/kube
```

1. Pick the options you want, and write a YAML or add some CLI arguments to our system.

Before moving ahead, **try** to complete this task. You can provision the resource with:

```
helm uninstall prometheus -n prometheus
helm install -f ./prometheus-values.yaml p
```

However, do not spend more than about 20 minutes trying to do this. You will run into a connection error that we want to talk about briefly that we haven't taught you yet how to overcome.

Information

On a complete side-note, there are multiple Helm charts for Prometheus that are considered "official", which makes Googling configuration options and debugging configuration options particularly tricky. We would suggest asking around for advice instead, as it will be easier to handle.

What Happened?

You likely got a connection error, where you were unable to connect to our Open-Telemetry connector service. However, it probably feels a bit strange. After all, normally we are able to connect each service together fairly easily, as the service DNS name is generally "servicename".

In this case, we have been foiled by namespaces. As a brief refresher, namespaces are how Kubernetes enforces some separation between what different groups see on the network. All of our provisioned resources have pods, but you do not see half of them as they are all in various namespaces.

While communication to services in the same namespace is easy, communicating between different namespaces is a bit trickier.

The general way you write the URL is the following:

```
SERVICENAME.NAMESPACE.svc.cluster.local:P(
```

As you can see, we've added a fair bit more to our DNS syntax, and we start with the service name as opposed to the namespace name to address our system. One benefit of this system, however, is that you can use it to be very clear when you are talking to something you do not control directly. If you connect to services you do and cannot control using this syntax, and then use the standard syntax for services you can connect to, you can set a fairly legible standard for how you connect and when you should consider debugging an interface instead of trying to reach an agreement.

With this information, you have enough to fill out the target, provided you know that the port number is **9464**. You will need to think carefully about the service name, and the namespace, but we covered an issue very similar to this in Project 2 Task 2.

Validation

If you have finished this task, you should be able to access new metrics on your Prometheus instance. In particular:

1. You get various metrics related to `asyncio`, as our FastAPI solution uses it for asynchronous processing.
2. You get various python-specific metrics, like the number of times your program switched from looking at one task to another with `process_runtime_cpython_context_switches_total`.
3. You get some configuration parameters that are always handy to log, like the number of threads being used by our llmserver with `system_thread_count{job="llmserver"}`

While using Istio would have given us some of the request-based metrics, the benefits of using something specialized to Python in this manner is that we can also get metrics that are Python-specific, and thus incredibly more useful.

Subtask 2: Django Instrumentation

With our success with FastAPI, you might think that the hard part is done. It would be nice for both of our Python services to be able to use this particular approach to metrics, as it doesn't involve editing the Docker image or anything.

Unfortunately, Django and this particular kind of instrumentation do not work well together. Django, with its focus on its own CLI that you need to manage, ends up missing the global settings when instrumented in this way, in a bug that hasn't been fixed for the last two years. While we could be sour that there is another large-standing bug in open software, it is the case of the field, as not everything can be well-maintained with developers who have the drive and time to maintain code they wrote years ago. Moreover, it lets us see how a more traditional approach to writing this code would be - one that more closely resembles the local non-cloud-native development you have done before this class. It also allows for some more flexibility, as you have the freedom to add custom metrics and other systems to make your life easier.

Without further ado, let's begin. As this is another process where it gets pretty hairy pretty quickly, we suggest letting us know if you think we missed some step in this process.

Dockerfile Changes

Firstly, we will need to change our Dockerfile. It will need the following:

1. It needs to use Python 3.10 as opposed to Python 3.7. Our existing image is too old for the new tech, and it is easy to run into many issues resulting from that.
2. You will need to change how you run the Django application in the Dockerfile.
   Currently, we do the following:

```
CMD python manage.py runserver 0.0.0.0:808
```

However, in order for us to use this kind of instrumentation, we instead need to call python through the `opentelemetry-instrument` command, by doing something like `opentelemetry-instrument python ...`.

Moreover, we will need to pass some options over to the `opentelemetry-instrument` command:

- Our service name should be `login`, so that we can find the new metrics we add quickly and easily.
- We must set our list of `metrics-exporter`s to be `otlp,console`. You will want to include both otlp and the console, as connecting this manual system to Prometheus can be a bit tricky.
- We must set our `exporter_otlp_endpoint` to be the path to our system, at port *4317*. Getting this correct can be challenging - we suggest using the shorter name for this pod as both deployments are in the same namespace.
- Lastly, set our Django file to include the `--noreload` flag. This is just helpful when we want our Django system to error out productively.
- You can refer to this link from opentelemetry documentation for an example of the same.

Requirements.Txt Changes

You will also need to import the relevant packages. If you forget to import any of these packages, the errors are silent, as the open telemetry tool will simply continue as if the program was not being instrumented.

Please add the following to the requirements.txt file we use in the build:

```
opentelemetry-sdk
opentelemetry-instrumentation-django
opentelemetry-distro[otlp]
opentelemetry-exporter-otlp-proto-http
requests
```

In theory, this is it. If you do this, and then simply re-deploy your system, you should be able to get some new metrics. However, you are likely to run into weird errors. The code is not non-deterministic, but the general understanding you can have about a system such as this is relatively low.

Some potential tips:

1. If you run into an error regarding the settings.py file, try setting the following environment variable from `manage.py` in Kubernetes as well.
   DJANGO_SETTINGS_MODULE: "task3.settings"

2. If you are getting frustrated please ask your questions. We expect it to be smooth sailing, but this task can be hard if you take it for granted.

Validation

Assuming you have built and re-deployed the image accordingly, all you need to do now is check that it worked.

Again, you have two options. You can either Curl the OpenTelemetry metrics endpoint, or you can use Prometheus directly to check if the new metrics are present.

The expected output of the curl command should be close to the following:

```
ttp_server_duration_milliseconds_bucket{ht
http_server_duration_milliseconds_bucket{h
http_server_duration_milliseconds_bucket{h
http_server_duration_milliseconds_bucket{h
http_server_duration_milliseconds_bucket{h
http_server_duration_milliseconds_bucket{h
http_server_duration_milliseconds_bucket{h
```

In the case of this service, here are a few interesting metrics that demonstrate the importance of these language-specific logging solutions:

The number of active requests the service is trying to handle.
A histogram of the duration different requests have gotten.

While it's significantly less than the automated installation, it is important that we have these metrics, so we can better separate the performance in any chart or any log we make.

Now that we have finished the Python-parts of this task, let's instrument the rest of our application.

Subtask 3: Instrumenting MariaDB

As databases are both really hard to scale effectively and really hard to optimize, getting metrics from whatever database you use is vital. Luckily, MariaDB provides easy to read metrics for this purpose.

All we should need to do is run a `helm upgrade` command, which will take the helm chart, the resources provisioned by it, and update them for this purpose:

```
helm upgrade mariadb-operator mariadb-oper
```

However, like most things in this class, what should be easy ends up being a bit more complicated. In order to get MariaDB hooked up, we will need to go through several steps.

Subtask 3.1: MariaDB Metrics Configuration

Simple upgrading the helm installation does not work as MariaDB's operator does not forcefully restart all of its pods. For this class, we will take the easier route of simply uninstalling and re-installing MariaDB:

```
helm uninstall mariadb-operator
helm install mariadb-operator mariadb-oper
  --set metrics.enabled=true
```

Once that is done, we will need to re-provision our CRDs. Delete the CRDs in any order you like, and then re-apply them in the order we discussed previously:

```
kubectl apply -f mariak8s/mariadb-config.y
kubectl apply -f mariak8s/mariadb-secret.y
kubectl apply -f mariak8s/mariadb-pod.yaml
kubectl apply -f mariak8s/mariadb-db.yaml
```

After doing this, if you look at `kubectl get svc`, you will see something like the following:

```
NAME
apigateway
kubernetes
llmservice
loginservice
mariadb-galera
mariadb-galera-internal
mariadb-galera-maxscale
mariadb-galera-maxscale-internal
mariadb-galera-metrics
mariadb-galera-primary
```

```
mariadb-galera-secondary
mariadb-operator-cert-controller-metrics
mariadb-operator-metrics
mariadb-operator-webhook
mariadb-operator-webhook-metrics
simplest-collector
simplest-collector-headless
simplest-collector-monitoring
```

Besides the sheer number of services in our application (and the fact that more than half are just the database...), we can see that we now have four metric services instead of the one we had previously:

mariadb-operator-metrics
mariadb-galera-metrics
mariadb-operator-webhook-metrics
mariadb-operator-cert-controller-metrics

What we would like to do is integrate these metrics into our Prometheus installation. Luckily, this process is the exact same of what we did in the first subtask:

1. Go to the service, and make sure that they have a metrics endpoint.
2. Confirm that the metrics seem accurate.
3. Update your values.yaml file, and restart Prometheus.
4. Run a couple of queries so there is fresh data for Prometheus to ingest.

Your second to last subtask for this project is to get this connected up. This shouldn't take much more than repeating the process we walked you through earlier.

Subtask 4: Instrumenting Istio

Unlike the first two subtasks, this is fairly tame. We are using Istio, which provides built-in metrics for all traffic flowing through the mesh.

Istio control plane (`istiod`) and Ingress Gateway expose Prometheus-compatible metrics by default.
- Check if `istiod` service exists: `kubectl get svc istiod -n istio-system`.
- Metrics port is usually `15014`.

Now, if we try to use our Gateway, we are still going to be unable to access the metrics, as they are not meant to be available on this port. Luckily, they are accessible

from the pod. You can check this using `kubectl port-forward`:

```
kubectl port-forward -n istio-system svc/i
```

If you go to http://localhost:15014/metrics , you will see the same metrics page we have gotten very familiar with over the past few weeks. So while we have our metrics page, it's not immediately clear what the best way to attach it is. The only way we attach things is through services, and currently the service attached to the gateway pod is not connected to that port.

Your task is to solve this issue. You should look at the LoadBalancer yaml you just installed.

Final Validation

At this point, you have managed to get all of the data that we can get for your application. You have managed to gather quite a lot of data, and have learned a bit of the many kinds of data available to you when you choose to use Cloud Native observability tools.

Make sure you are under the root directory of the repository, and submit with:

```
./submitter -t task3
```

← Previous Page Next Page →

Report an Issue? © 2026 Copyright Sail() Platform